F.CSM302 ҮЙЛДЛИЙН СИСТЕМ

БИЕ ДААЛТ 1 — СУДАЛГАА, ХЭРЭГЖИЛТ, ТУРШИЛТЫН ТАЙЛАН

Linux Kernel Module for Task Information

Сурах бичиг: Silberschatz, Operating System Concepts (10th ed.), Chapter 3 Processes — Programming Project 2

Хуваарилалтын код: **-<цаг>-2**

UI дүрслэл: Task hierarchy

Гишүүн 1 — _____ (B_____)

Гишүүн 2 — _____ (B_____)

Гишүүн 3 — _____ (B_____)

Гишүүн 4 — _____ (B_____)

Гишүүн 5 — _____ (B_____)

GitHub: https://github.com/temkagg1117-lab/os-bd1

Хураангуй

Асуудал. Үйлдлийн системийн kernel дотор task (process)-ийн удирдлагын бүтэц болох `task_struct`-ийг хэрэглэгчийн орон зайгаас аюулгүй, давтан туршиж болохоор харах хэрэгтэй. **Шийдэл.** `/proc/pid` файлаар pid хүлээн авч command, pid, state-ийг (номын шаардлага) болон parent, ancestors, children-ийг (Task hierarchy UI-д) буцаадаг Linux kernel module, мөн түүнийг бодит цагт SVG мод болгон зурдаг web UI. **Туршилт.** Shell дээр 9 бүлэг үндсэн ба захын тест, backend-ийн 18 автомат тест (mock горимд 18/18 OK). Бодит VM дээрх дүнг §8-д бүртгэнэ.

1. Удиртгал

Зорилго. Номын Project 2-ыг бүрэн хэрэгжүүлж, хэрэглэгч өөр PID, гүн, хязгаараар туршиж болох Task hierarchy UI-тэй холбох. **Ач холбогдол.** Process control block, process tree, kernel–user харилцаа (/proc) гэсэн Chapter 3-ын гол ойлголтуудыг нэг ажилд хамааруулж, kernel дэх бодит өгөгдөл дээр ажиглах боломж олгоно. **Хамрах хүрээ.** Linux 5.x/6.x, x86-64; хамгийн багадаа command, pid, state; нэмэлтээр parent, ancestors, children. Thread-ийн дотоод (tid) түвшин, namespace-ийн бүрэн дэмжлэг хамаарахгүй.

2. Сурах бичгийн шаардлага

Project 2-ын шаардлагыг (Silberschatz, 10th ed., P-15…P-17) дараах байдлаар хангасан:

Номын шаардлага	Хэрэгжилт
`echo "1395" > /proc/pid` — pid бичих	`ti_write()`: kmalloc → copy_from_user → `kstrtol()` → kfree; зөв утгыг глобал хувьсагчид хадгална
`cat /proc/pid` — command, pid, state унших	Эхний мөр яг номын формат: `command = [bash] pid = [1395] state = [1]`
`find_vpid()` + `pid_task(…, PIDTYPE_PID)`	`ti_build_report()` дотор
Хүчингүй pid → pid_task NULL → read 0 буцаах	NULL үед 0 буцаана (`simple_read_from_buffer`-д len=0), dmesg-д тэмдэглэнэ
Санах ойн зөв удирдлага (kmalloc/kfree)	Бүх замд kfree; алдааны салаанд бас чөлөөлнө
Chapter 2-ын /proc entry	`proc_create()` / `remove_proc_entry()`

3. Онолын үндэс

Process control block. Linux-т PCB нь `struct task_struct` (<linux/sched.h>). Үүнд `comm` (командын нэр, 16 байт), pid/tgid, state, `real_parent`, `children`, `sibling` талбарууд байна.

Process state. Номын Figure 3.2-ын new/ready/running/waiting/terminated төлвүүд Linux-д `TASK_RUNNING(0)`, `TASK_INTERRUPTIBLE(1)`, `TASK_UNINTERRUPTIBLE(2)`, `__TASK_STOPPED(4)`, `__TASK_TRACED(8)` болон exit_state (`EXIT_ZOMBIE`, `EXIT_DEAD`) байдлаар илэрнэ. Kernel 5.14-өөс талбарын нэр `state` → `__state` болсон тул хувилбарын нөхцөлөөр зохицуулав.

Process tree. fork() бүрээр parent–child холбоо үүснэ; init (pid 1) бол үндэс (Figure 3.8). Kernel процессуудыг `children`/`sibling` холбоост жагсаалтаар хадгалдаг. Энэ ажилд parent-ийг `real_parent`-ээс, child-ийг `for_each_process()` дундаас `real_parent->tgid == target->tgid` нөхцөлөөр олно (tasklist_lock нь module-д export хийгдээгүй тул RCU-аар уншина).

RCU. `rcu_read_lock()` дотор task_struct-ийг унших нь аюулгүй; хугацаа уртасгахгүйн тулд kmalloc-ийг lock-оос өмнө хийж, доторх нь sleep хийхгүй (scnprintf-ээр урьдчилан хуваарилсан buffer-т бичнэ).

/proc. Хэрэглэгчийн орон зайд kernel өгөгдлийг файл маягаар гаргах механизм. Бичих үед `copy_from_user`, унших үед `simple_read_from_buffer` (→ copy_to_user) ашиглана.

4. Системийн зохиомж

Компонент ба өгөгдлийн урсгал:

Давхарга	Файл	Үүрэг
Browser UI	ui/static/index.html	PID/гүн оролт, SVG мод, алдааны мэдээлэл
Backend (HTTP)	ui/server.py	/api/tree, /api/query; мод бүрийн node-д write pid → read report; Lock-оор хамгаална
/proc интерфейс	src/ti_main.c	/proc/pid: write (шалгалт), read
Walker	src/ti_walk.c	task_struct → тайлан (RCU)
Kernel	task_struct	Бодит процессын мэдээлэл

Гаралтын формат (мөр бүр `key = [value]`):

```
command = [bash] pid = [1395] state = [1]          <- номын формат
state_name = [TASK_INTERRUPTIBLE]
ppid = [1200] parent_command = [sshd]
nchildren = [2]
ancestor: pid = [1] command = [systemd] state = [1]    (үндэс эхэнд)
child: pid = [1400] command = [sleep] state = [1]
truncated = [children]                                  (>128 child үед)
```

UI мод үүсэх дараалал. UI сонгосон PID-г асууж ancestor-уудыг авна → тэр node-ийн `child:` мөр бүрээр дахин асууж гүн (depth) хүртэл доош нь өргөтгөнө (node-ийн дээд тоо хязгаартай). Тиймээс зурагдсан мод бүхэлдээ module-ийн бодит хариултаас баригдана.

5. Хэрэгжилт

5.1 /proc интерфейс (ti_main.c). 0666 эрхтэй `proc_create(proc_name,…)`; kernel ≥5.6-д `struct proc_ops`, түүнээс өмнө `file_operations`. Write-ийн шалгалт: урт 1..16; `strim` + `kstrtol`; утга 1..PID_MAX_LIMIT; алдаанд `-EINVAL` ба `pr_warn`. Глобал pid-г mutex хамгаална.

```
rv = kstrtol(strim(k_mem), 10, &val);
if (rv || val <= 0 || val > PID_MAX_LIMIT) return -EINVAL;
mutex_lock(&ti_lock); ti_pid = val; mutex_unlock(&ti_lock);
```

5.2 Walker (ti_walk.c). `pid_task(find_vpid(pid), PIDTYPE_PID)`; NULL бол 0. Ancestor-уудыг ≤64 шатлал дээш алхаж цуглуулаад үндэс эхэнд хэвлэнэ; child-ийг ≤128 мөр. Бүх зүйл нэг `rcu_read_lock` дотор.

```
task = pid_task(find_vpid(pid), PIDTYPE_PID);
if (!task) { rcu_read_unlock(); return 0; }
for_each_process(p)
    if (p != task->group_leader && rcu_dereference(p->real_parent)->tgid == task->tgid) ...
```

5.3 Backend ба UI. Python stdlib HTTP сервер 127.0.0.1-д; API: `/api/query`, `/api/tree`, `/api/spawn` (туршилтын sleep мод), `/api/cleanup`. Алдаа: BAD_PID(400), NO_SUCH_PID(404), MODULE_NOT_LOADED(503), PERMISSION(403). `--mock` горим нь module-гүй машин дээр UI үзэх зориулалттай бөгөөд UI дээр шар тэмдэглэгээтэй.

5.4 GitHub ажиллагаа. (багийнхан бөглөнө)

Гишүүн	Салбар	Commit тоо	PR / Review	Issue
1	m1-proc-interface	__	__	__
2	m2-walker	__	__	__
3	m3-backend	__	__	__
4	m4-ui	__	__	__
5	m5-tests-docs	__	__	__

6. Туршилт ба UI дүрслэл

UI дээр өөрчилж турших зүйлс ба ажиглах зүйл:

Өөрчлөх	Ажиглах
PID (1, $$, 2, өөр process)	Ancestor гинж дээрээс доош; сонгосон node хар хүрээтэй
Depth 0..6, node max 1..500	Descendant-ын гүн; хязгаарт хүрвэл тэмдэглэл (+ тэмдэг, “мод бүрэн биш”)
“Мод үүсгэх” (гүн 1..3, салаа 1..4)	Шинэ бодит process мод; `ps -ef --forest`-той харьцуулах
Автомат шинэчлэл (2 сек)	Процесс `kill` хийхэд мод өөрчлөгдөх (бодит цаг)
Node дээр дарах	Тэр PID-ээр дахин асуулт; drill-down
Буруу оролт (abc, 0, 4194303)	Улаан алдааны мэдээлэл, дүрслэл эвдрэхгүй
Reset	Бүх оролт, зураг цэвэрлэгдэнэ

Өнгө: ногоон=RUNNING(0), цэнхэр=INTERRUPTIBLE(1), улбар шар=UNINTERRUPTIBLE(2), саарал=STOPPED/TRACED(4,8), улаан=ZOMBIE/DEAD.

7. Туршилтын арга

Орчин. Ubuntu 22.04/24.04 VM (kernel: _____), gcc, make, linux-headers, Python 3. **Өгөгдөл.** Хатуу кодлосон ганц өгөгдөл байхгүй: $$, pid 1, sleep процессууд, `/api/spawn`-аар үүссэн мод, тогтмол буруу утгууд. **Автомат.** `sudo ./tests/run_module_tests.sh` (бодит module), `python3 tests/test_ui_api.py [--module]` (backend). **Давтах.** Үр дүн бүрийг Manual-д заасан тушаалаар давтан шалгаж болно. Дэлгэрэнгүй: `tests/test_cases.md`.

8. Үр дүн

8.1 Backend автомат тест (mock горим). `python3 tests/test_ui_api.py` → **Ran 18 tests … OK** (T01–T04, E01–E09, H01–H03, S01–S02). Mock нь module-ийн гаралтыг `/proc/<pid>/stat`-аас дууриадаг тул энэ нь backend/parser/UI логикийн баталгаа бөгөөд **kernel module-ийн баталгаа биш**.

8.2 Бодит kernel module (VM). Доорх хүснэгтийн “Бодит” баганыг `sudo ./tests/run_module_tests.sh`-ийн гаралтаар бөглөнө:

ID	Оролт	Хүлээгдсэн	Бодит
T1	echo $$ > /proc/pid; cat /proc/pid	command = [bash] pid = [N] state = [n]	_____
T2	pid 1	pid = [1], ancestor байхгүй	_____
T3	ppid шалгалт	/proc/$$/stat-ийн ppid-тэй тэнцүү	_____
T4	2 sleep & үүсгэх	2 child: мөр	_____
T5	kill -STOP sleep	state = [4]	_____
T6	echo 4194303 > /proc/pid	read = 0 byte	_____
E1-E4	abc, 0, -5, 99999999999, хоосон, 12abc	write амжилтгүй (EINVAL), pid өөрчлөгдөхгүй	_____
R1	rmmod / insmod	/proc/pid устаж, дахин үүснэ	_____
K1	dmesg	oops/BUG/RCU warning байхгүй	_____

8.3 Screenshot. (1) `cat /proc/pid` гаралт; (2) UI дээрх мод (pid 1, depth 2); (3) “Мод үүсгэх”-ийн дараах мод ба `ps -ef --forest` зэрэгцүүлсэн; (4) алдааны мэдээлэл; (5) dmesg. — [багийнхан бодит screenshot-оо энд оруулна]

9. Хэлэлцүүлэг

• Номын загвар ганц глобал pid хадгалдаг тул зэрэг ажиллах хэрэглэгчид pid-г дарж болно; backend write+read-ийг lock-оор хамгаалсан боловч module түвшинд per-open төлөв хэрэгжүүлбэл илүү зөв.

• Children-ийг for_each_process-оор олох нь O(N) (N = нийт процесс); мод бүрийн node бүрт дахин асуулт өгдөг тул том системд node-ийн дээд хязгаар шаардлагатай.

• Асуулт хооронд процесс гарч болох тул UI нь алдагдсан child-ийг бүдэг (gone) node-оор харуулна — мод нь агшин зуурын зураг, атомар биш.

• UI-д ZOMBIE-г өнгөөр ялгах мэдээлэл зөвхөн сонгосон task-ийн state_name-д байна (child: мөрд зөвхөн state тоо).

• PID namespace/container дотор task_pid_vnr нь дуудагчийн namespace-ээр харагддаг; гаднах parent 0 гэж харагдана.

• Kernel хувилбарын нийцэл (5.6 proc_ops, 5.14 __state) нь LINUX_VERSION_CODE-ээр шийдэгдсэн; зөвхөн багийн VM-ийн хувилбар дээр баталгаажсан.

10. Дүгнэлт

Номын Project 2-ын бүх шаардлага (pid бичих, command/pid/state унших, хүчингүй pid үед 0, kstrtol, kmalloc/kfree) болон Task hierarchy UI (parent, ancestors, children мод) хэрэгжсэн. Backend-ийн 18 автомат тест давсан. Module-ийн бодит ажиллагааг §8.2-ын хүснэгт болон screenshot-оор нотлоно. Цаашид: per-open төлөв, Thread (tid) түвшний харагдац, task state-ийн өнгө бүх node-д.

Ашигласан материал

• A. Silberschatz, P. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018 — Ch.3, Programming Projects P-15…P-17.

• Linux kernel documentation: proc_ops, RCU (Documentation/RCU), include/linux/sched.h.

• man 5 proc; man 1 ps.

• **AI ашигласан тэмдэглэл:** Анхны код, UI, тест, баримтын ноорогыг Claude (Anthropic) AI туслагчаар бэлтгэсэн; баг бүх кодыг уншиж, VM дээр build/run/test хийж баталгаажуулсан (README-д мөн тэмдэглэсэн). Бэлэн сан ашиглаагүй (Python stdlib, kernel headers).